

Html interview question

What is html?

Html stands for hipper text markup language.

It is used for describing the content and structure of a web page.

What are semantic tags in html?

Tags which describe its meaning to a browser for the developer.

And also helps in searching and optimization.

example: table, article, footer, header, etc...

what is difference between tag and element in html?

Tags: it refers to opening and closing tag.

Element: it also includes opening and closing tags but also with the content inside it. Collectively we called it element.

Attributes: it provides additional information about

Element i.e it helps elements to how should it behave and appear .it always placed in opening tag.

Common HTML Attributes:

href Attribute

The href attribute is used with the <a> tag to define a hyperlink. It specifies the URL of the page the link goes to:

```
<a href="https://www.w3schools.com">Visit W3Schools</a>
```

src Attribute

The src attribute is used with the tag to embed an image in an HTML page. It specifies the path to the image to be displayed:

```

```

alt Attribute

The alt attribute provides an alternate text for an image if the image cannot be displayed. This is useful for accessibility and when the image fails to load:

```

```

style Attribute

The style attribute is used to add inline CSS styles to an element, such as color, font, size, and more:

```
<p style="color:red;">This is a red paragraph.</p>
```

title Attribute

The title attribute defines extra information about an element, which is displayed as a tooltip when you mouse over the element:

```
<p title="I'm a tooltip">This is a paragraph.</p>
```

Difference between div and span?

Div: it is block level element.

Span: is an inline level element.

In HTML, the `<div>` and `<span>` tags are used for structuring and styling content. `<div>` creates block-level containers for grouping elements, while `<span>` creates inline containers for styling specific portions of text or elements within a block-level container.

Difference between inline and block level elements

Block elements: They consume the entire width available irrespective of their sufficiency. They always start in a new line and have top and bottom margins. It does not contain any other elements next to it.

- [<h1>-<h6>](#) : This element is used for including headings of different sizes ranging from 1 to 6.
- [<div>](#): This is a container tag and is used to make separate divisions of content on the web page.
- [<hr>](#): This is an empty tag and is used for separating content by horizontal lines.
- [](#): This tag is used for including list items of an ordered or unordered list.
- [](#): This tag is used to make an unordered list.
- [](#): This tag is used to make an ordered list.
- [<p>](#): This tag is used to include paragraphs of content in the webpage.
- [<table>](#): This tag is used for including the tables in the webpage when there is a need for tabular data.

Inline elements: Inline elements occupy only enough width that is sufficient to it and allows other elements next to it which are inline. Inline elements don't start from a new line and don't have top and bottom margins as block elements have.

Examples of Inline elements:

- [<a>](#): This tag is used for including hyperlinks in the webpage.

- [
](#): This tag is used for mentioning line breaks in the webpage wherever needed.
- [<script>](#) : This tag is used for including external and internal JavaScript codes.
- [<input>](#): This tag is used for taking input from the users and is mainly used in forms.
- [](#): This tag is used for including different images in the webpage to add beauty to the webpage.
- [](#): This is an inline container that takes necessary space only.
- [](#): This tag is used in places where bold text is needed.
- [<label>](#): The tag in HTML is used to provide a usability improvement for mouse users i.e, if a user clicks on the text within the <label> element, it toggles the control.

Differences between replaced inline elements and non-replaced inline level elements?

replaced inline: elements which are that element which came from URL like: img, video, iframe, tags.

That's why in case of replaced inline elements we can change its height and width.

non-replaced inline level elements:

all other elements that's height width cannot be changed ,they start from the new line are called non-replaced .

ex: label,,<hr> ,<input >etc.

why do we include <!Doctype html > In starting of html?

Doctype in HTML: HTML Doctype is most often written at the very first element of the entire HTML document. It remains wrapped inside angle brackets but it is not a tag. It is a statement or declaration. Doctype stands for Document Type. It is a statement to declare the type of the document. With the help of this statement, the developer let the browser know that the following document is an HTML document.

Q>diff b/w & tags?

: it is used to bold the text ,but its just for styleing.

** :** it is used to bold the text and tells the browser its important.

Q>diff b/w <i> & tags?

<i>: it makes the text slanted(italic),just for styleing.

: it makes the text slanted(italic),and adds emphasis when reading allowed.

Q> diff b/w absolute & relative url's?

Absolute url: it is complete web address with protocol(http:// , https://) ,domain name, and file path.

Ex:- http://www.domain.com/ file_path(logo.png)

Relative url: it is partial web address that link with page or file based on the current pages location.

What is the latest version of HTML?

Current Version of HTML 5

The **current version of HTML** is **HTML5**. It is the fifth iteration of the HyperText Markup Language and serves as the standard for structuring and presenting content on the web.

HTML5 introduced significant improvements over its predecessors, including:

- **Semantic Elements:** New tags like <header>, <footer>, <section>, and <article> for better document structure.
- **Multimedia Support:** Native support for <audio> and <video> elements, eliminating the need for external plugins like Flash.
- **Offline Storage:** Features like localStorage and sessionStorage for storing data locally on the user's device.
- **Graphics and Interactivity:** Support for <canvas> and **SVG** for drawing graphics and animations directly in the browser.
- **Geolocation API:** Enables websites to access a user's geographical location.
- **Improved Mobile Compatibility:** Designed to be more mobile-friendly and efficient.

HTML5 also simplifies syntax, improves performance, and enhances flexibility, making it the foundation for modern web development.

Q>Local Storage vs Session Storage in HTML

Both **Local Storage** and **Session Storage** are part of the Web Storage API in HTML5, used to store data on the client-side

Feature	Local Storage	Session Storage
Data Lifespan	Stays even after closing the browser	Gets deleted when the tab or browser is closed
Storage Capacity	~5MB to 10MB (depending on browser)	~5MB
Persistence	Permanent until manually deleted	Temporary — only for that session
Accessibility	Available across all tabs and windows of the same origin	Only available in the tab it was created in
Sent to Server?	No (stays on user's machine only)	No
Use Case	Saving preferences, themes, cart items, login state	Temporary forms, one-time data entry, wizards
Example Metaphor	Like notes in a notebook on your desk	Like text on a whiteboard that gets erased later

.

Q> diff b/w id and class attributes in html?

Both are used to accessing and styling html element .

But have diff purpose & distinct characteristics:

Attribute	Id	Class
Purpose	Identifies one unique element	Applies style/behavior to multiple elements
Usage in CSS	#special { color: blue; }	.highlight { color: green; }
Uniqueness	Must be unique on the page	Can be shared across elements
Example	<p id="special">...</p>	<p class="highlight">...</p>
When to Use	For targeting a specific section (e.g., navbar)	For styling rep

How to use css in html ?

External CSS

- . CSS is written in a separate .css file
- . Linked using <link> inside the <head>

Internal CSS

- . CSS is written inside a <style> tag within the <head> of the HTML file
- . Affects only that specific HTML page

Inline CSS

- . CSS is added directly to an HTML element using the style attribute
- . Good for quick or one-time styles

Q. What are **Meta Tags**? What are the **5 types** of meta tags?



1. Character Encoding:

This helps browsers to interpret the different characters in the document. The `charset="UTF-8"` attribute ensures that characters from different languages will be displayed correctly in the browser.

2. Responsive Design:

viewport meta tag is crucial for creating mobile-friendly, responsive web designs for various screen sizes.

3. Description for SEO:

This meta tag provides description of the content of the page. Search engines may use this for search results.

4. Keywords for SEO:

This meta tag used to be important for search engines, but it's now of less importance.

5. Author Info:

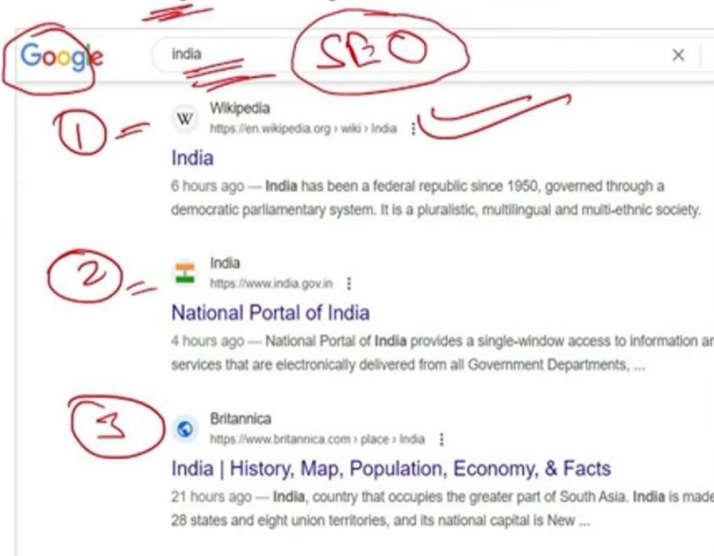
This meta tag can be used to specify the author of the document.

```
<html>
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Interview Happy</title>
    <meta name="description" content="Technical interview preparation" />
    <meta name="keywords" content=".NET, Angular, React, JS, HTML" />
    <meta name="author" content="Happy Rawat" />
  </head>
```

Q. What is **SEO**? What are **5 HTML Best Practices** for SEO?



❖ **SEO**(Search Engine Optimization) is the practice of optimizing a website to improve its **visibility** and **ranking** in search engine results.



5 HTML Best Practices for SEO

1. Use Semantic HTML Elements

2. Optimize Page Titles

3. Use Meta Description Tags

4. Use Proper Heading Tags

5. Optimize Image Alt Attributes

6. What is an Anchor tag in HTML?

An anchor tag in HTML, represented by the `<a>` element, is used to create hyperlinks, which are fundamental to web navigation.

These hyperlinks can link to various resources, such as other web pages, files, email addresses, or locations within the same page.

11. What is HTML5?

HTML5 is the fifth and latest major version of the HyperText Markup

Language (HTML), which is the standard language for creating and structuring content on the web. It was officially published as a W3C Recommendation on October 28, 2014. HTML5 introduces new elements, attributes, and behaviors, providing more powerful tools for web developers to create complex and dynamic web pages and applications.

13. What is an Image Map?

An image map in HTML is a graphical element that allows you to define multiple clickable areas (or hotspots) on a single image. Each of these areas can act as a hyperlink, directing the user to different destinations when clicked.

15. What is a 'Marquee' Tag in HTML?

The <marquee> tag in HTML is used to create scrolling text or images within a web page. Introduced in early versions of HTML, the <marquee> element allows content to move across the screen horizontally or vertically, attracting attention or adding a dynamic visual effect.

14. Why is the Embed Tag Used in HTML?

The <embed> tag is a versatile and powerful tool that allows you to integrate external content directly into your web pages. This tag is incredibly useful for embedding various types of media and interactive content .

16. How do we add a copyright symbol on a browser page?

To add a copyright symbol on a browser page, you simply include a special HTML entity like © or © within your text, which will render the c symbol when viewed in a browser, ensuring your content is properly marked with the copyright notice.

21. How are active links different from normal links?

Different browsers determine an active link through different criteria. Some identify it when the mouse cursor hovers over the link, while others recognize it when the link is in focus. A link without the mouse cursor over it is regarded as a normal link.

22. What is the difference between HTML and XHTML?

Feature	HTML (HyperText Markup Language)	XHTML (Extensible HyperText Markup Language)
Syntax Rules	More lenient, allowing flexibility in coding practices.	Stricter, requiring well-formed code following XML rules.
Closing Tags	Not all tags require closing tags.	All tags must be properly closed.
Case Sensitivity	Case-insensitive; tags can be written in uppercase or lowercase.	Case-sensitive; all tags must be written in lowercase.
Attributes	Attributes can be unquoted.	Attributes must be quoted.
Error Handling	More forgiving with errors.	Less forgiving; errors must be corrected for the code to work.
Compatibility	Less consistent but widely supported.	More consistent and compatible with XML parsers.

25. What is a datalist tag?

The `<datalist>` tag offers an autocomplete feature in HTML files, allowing users to add a form with predefined options. When used in conjunction with an input tag, it makes way for filling forms.

28. What is an SVG tag in HTML?

SVG stands for Scalable Vector Graphics in HTML. HTML SVG is a modularized language used to depict visuals using XML. XML is used to articulate two-dimensional vector and hybrid vector/raster graphics.

Actions of SVG images are delineated in text files containing XML.

Difference Between HTTP and HTTPS

HTTP (HyperText Transfer Protocol) and HTTPS (HyperText Transfer Protocol Secure) are protocols used for transferring data over the web. The key distinction lies in the security features provided by HTTPS, which makes it more secure than HTTP.

what are self closing tags in html

1.
No Closing Tag Needed: They combine the opening and closing functionality into a single tag.
2. **Syntax:** They end with a slash (/) before the closing angle bracket (>), though this is optional in HTML5.
 - Example: ``
3. **Purpose:** Used for elements that represent empty or solitary content.

Difference Between Canvas and SVG

Canvas and SVG are two popular technologies for creating graphics on the web, but they differ significantly in their approach, use cases, and capabilities.

Canvas is a **bitmap-based** graphics system introduced in HTML5. It provides a **pixel-manipulation API** for drawing graphics programmatically using JavaScript. Canvas is **imperative**, meaning you describe how to draw step-by-step. It is ideal for **dynamic, high-performance rendering** such as games, animations, or real-time visualizations. However, it lacks scalability, as resizing can cause pixelation, and it does not integrate with the DOM, making individual elements unmanageable. SVG, on the other hand, is **vector-based** and uses an XML format to describe graphics. It is **declarative**, meaning you define what to draw, and the browser renders it. SVG elements are part of the DOM, allowing them to be styled with CSS and manipulated with JavaScript. SVG excels

in **scalability**, maintaining quality at any resolution, and is suitable for **static or moderately interactive graphics** like charts, icons, and illustrations.

What is the other major difference apart from div being a block element and span being an inline element?

We can not give height and width to inline level elements. Span

HTML <template> Tag?

The <template> tag in HTML is a special container used to hold HTML content that is not displayed when the page initially loads. It is primarily used to store reusable HTML fragments that can be dynamically rendered or cloned into the DOM using JavaScript.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Template Example</title>
</head>
<body>
  <template id="card-template">
    <div class="card">
      <h2></h2>
      <p></p>
    </div>
```



```
</template>

<div id="container"></div>

<script>

  const template = document.getElementById('card-template');
  const container = document.getElementById('container');

  // Clone the template content
  const clone = template.content.cloneNode(true);
  clone.querySelector('h2').textContent = 'Dynamic Title';
  clone.querySelector('p').textContent = 'This content was added dynamically!';

  // Append the cloned content to the container
  container.appendChild(clone);
</script>
</body>
</html>
```

[Example of a Void Element in HTML?](#)

A **void element** in HTML is an element that does not have any content and does not require a closing tag. These elements are self-contained and are typically used for structural or functional purposes in a webpage.

- `<br>` – Line break
- `<img>` – Image

- `<hr>` – Horizontal rule
- `<input>` – Input field
- `<meta>` – Metadata
- `<link>` – Link to external resources (like CSS)
- `<source>` – Media source for `<audio>` or `<video>`
- `<area>` – Clickable area in an image map
- `<col>` – Column in a table
- `<embed>` – Embedded content (like videos)
- `<track>` – Text tracks for media

Global Attributes in HTML ?

Global Attributes in HTML are attributes that can be applied to almost all HTML elements. Here are some commonly used ones:

1. **id**: Specifies a unique identifier for an element.
2. **class**: Assigns one or more class names for styling or scripting.
3. **style**: Defines inline CSS styles for an element.
4. **title**: Provides additional information about an element, often displayed as a tooltip.
5. **lang**: Specifies the language of the element's content.
6. **data- (Custom Data Attributes)***: Allows embedding custom data attributes for scripting purposes.

7. **hidden**: Indicates that the element is not yet, or is no longer, relevant.
8. **tabindex**: Specifies the tab order of an element.
9. **draggable**: Specifies whether an element is draggable.
10. **contenteditable**
: Indicates whether the content of an element is editable.
11. **accesskey**:
Defines a shortcut key to activate or focus an element.
12. **dir**: Specifies
the text direction (e.g., "ltr" for left-to-right or "rtl" for right-to-left).

These attributes enhance the functionality, accessibility, and styling of HTML elements.

GET and POST ?

GET and **POST** are two commonly used HTTP methods in HTML for sending data between a client (browser) and a server. Here's a concise comparison:

GET Method

1. **Purpose**: Used to retrieve data from the server.
2. **Data Transmission**: Sends data appended to the URL as query parameters (e.g., example.com/page?name=John).
3. **Visibility**: Data is visible in the URL, making it less secure for sensitive information.

4. **Length:** Limited amount of data can be sent due to URL length restrictions.
5. **Caching:** GET requests can be cached and bookmarked.
6. **Use Case:** Ideal for fetching data without side effects, like search queries or reading resources.

POST Method

1. **Purpose:** Used to send data to the server to create or update resources.
2. **Data Transmission:** Sends data in the request body, not visible in the URL.
3. **Visibility:** Data is hidden from the URL, making it more secure for sensitive information.
4. **Length:** No restrictions on the amount of data sent.
5. **Caching:** POST requests are not cached or bookmarked.
6. **Use Case:** Suitable for submitting forms, uploading files, or sending sensitive data.

Form validation ?

Form validation in HTML is the process of ensuring that the data entered into a form by a user meets specific requirements before it is submitted to the server. It helps improve data accuracy, user experience, and security by preventing incomplete or incorrect data from being processed.

Types of Form Validation in HTML

1. Client-Side Validation:

- Performed in the user's browser before the form is submitted.
- Uses HTML attributes or JavaScript to validate input fields.
- Provides instant feedback to users, such as error messages for missing or invalid inputs.

2. Server-Side Validation:

- Performed on the server after the form is submitted.
- Ensures data integrity and security, even if client-side validation is bypassed.

HTML-Based Validation Features

HTML5 introduced several built-in attributes for form validation, such as:

- **required:** Ensures the field cannot be left empty.

Html

```
<input type="text" name="username" required>
```

- **pattern:** Specifies a regular expression that the input must match.

Html

```
<input type="text" name="phone" pattern="[0-9]{10}" title="Enter a 10-digit phone number">
```

- **type:** Validates input based on the type (e.g., email, number, url).

Html

`<input type="email" name="email">`

- **min, max, maxlength, minlength:** Define numeric or character limits.

Html

`<input type="number" name="age" min="18" max="60">`

- **step:** Specifies valid intervals for numeric inputs.

Html

`<input type="number" name="price" step="0.01">`

step defines **how much the value can increase or decrease at a time.**

Here, step="0.01" means:

- The input **allows decimal values up to 2 decimal places**
- The value will change by **0.01** when you:
 - Click the up/down arrows
 - Use keyboard arrows (↑ ↓)

- **novalidate:** Disables HTML validation for the form.

Html

`<form novalidate>`

Advantages of HTML Form Validation

- No need for additional JavaScript for basic validation.
- Faster feedback to users.

- Reduces server load by catching errors early.

For more advanced validation, JavaScript can be used to customize error messages or validate complex conditions.

readonly Attribute ?

- **Purpose:** Makes the input field non-editable while still allowing the user to see its value.
- **Usage:** Useful for displaying data that should not be modified.
- **Behavior:** The field's value can be submitted with the form.
- **Example:**

Html

Copy code

```
<input type="text" value="Read-only text" readonly>
```

<label> tag in HTML??

The **<label>** tag in HTML is used to define a label for an input element in a form. Its primary purpose is to improve **accessibility** and **usability** by associating descriptive text with form controls like text fields, checkboxes, radio buttons, etc.

Key Features of the <label> Tag:

1. **Accessibility:** It helps screen readers identify the purpose of form controls, making forms more accessible to users with disabilities.

2. **Clickable Area:** When a user clicks on the text within the `<label>` tag, it automatically focuses or activates the associated input element. This increases the clickable area, which is especially helpful for small elements like checkboxes or radio buttons.
3. **Association:** The `<label>` tag can be associated with an input element in two ways:
 - **Using the for attribute:** The for attribute's value should match the id of the input element.

Html

Copy code

```
<label for="username">Username:</label>
```

```
<input type="text" id="username" name="username">
```

- **Wrapping the input element:** The input element can be nested inside the `<label>` tag.

Html

Copy code

```
<label>
```

```
  <input type="checkbox" name="subscribe"> Subscribe to newsletter
```

```
</label>
```

Benefits:

- Enhances user experience by making forms easier to interact with.

- Improves form navigation for users relying on assistive technologies.
- Provides a larger target area for touch devices.

In summary, the <label> tag is a small but powerful tool for creating user-friendly and accessible forms!

Autocomplete in HTML?

Autocomplete in HTML is an attribute used to control whether a browser should automatically fill in values for form fields based on the user's previous inputs. It enhances user experience by saving time and reducing repetitive typing.

```
<form action="/submit" method="post">
```

```
<label for="email">Email:</label>
```

```
<input type="email" id="email" name="email" autocomplete="on">
```

```
<label for="password">Password:</label>
```

```
<input type="password" id="password" name="password"  
autocomplete="off">
```

```
<button type="submit">Submit</button>
```

Semantic Elements in HTML?

Semantic elements in HTML are those that clearly describe their meaning in a human- and machine-readable way. These elements provide better structure and meaning to web content, improving accessibility, SEO, and overall user experience.

Key Semantic Elements

<header>

The <header> element represents a container for introductory content or a set of navigational links. It typically contains one or more heading elements (<h1> - <h6>), a logo or icon, and authorship information. Multiple <header> elements can be used in a single document.

```
<header>
```

```
<h1>Company A</h1>
```

```
<nav>
```

```
<ul>
```

```
<li><a href="/home">Home</a></li>
```

```
<li><a href="/about">About</a></li>
```

```
<li><a href="/contact">Contact</a></li>
```

```
</ul>
```

```
</nav>
```

```
</header>
```

<footer>

The <footer> element defines a footer for a document or section. It typically contains authorship information, copyright information, contact information, sitemap, and related documents.

```
<footer>
```

```
<p>Author: John Doe</p>
```

```
<p><a href="mailto:john@example.com">john@example.com</a></p>
```

```
</footer>
```

<article>

<article>

The <article> element specifies independent, self-contained content that should make sense on its own and can be distributed independently from the rest of the site. Examples include blog posts, news articles, and user comments^{[12](#)}.

```
<article>
```

```
<h2>Google Chrome</h2>
```

```
<p>Google Chrome is a web browser developed by Google, released in 2008. Chrome is the world's most popular web browser today!</p>
```

```
</article>
```

<section>

The <section> element defines a thematic grouping of content, typically with a heading. It is used to group related content together^{[12](#)}.

`<section>`

`<h1>WWF</h1>`

`<p>The World Wide Fund for Nature (WWF) is an international organization working on issues regarding the conservation, research, and restoration of the environment.</p>`

`</section>`

`<nav>`

The `<nav>` element defines a set of navigation links. It is intended only for major blocks of navigation links^{[12](#)}.

`<nav>`

`<a href="/html/">HTML</a> |`

`<a href="/css/">CSS</a> |`

`<a href="/js/">JavaScript</a> |`

`<a href="/jquery/">jQuery</a>`

`</nav>`

`<aside>`

The `<aside>` element defines content aside from the content it is placed in, like a sidebar. The content should be indirectly related to the surrounding content^{[12](#)}.

`<aside>`

<h4>Epcot Center</h4>

<p>Epcot is a theme park at Walt Disney World Resort featuring exciting attractions, international pavilions, award-winning fireworks, and seasonal special events.</p>

</aside>

<figure> and <figcaption>

The <figure> element specifies self-contained content, like illustrations, diagrams, photos, etc. The <figcaption> element defines a caption for the <figure> element^{[12](#)}.

<figure>

<figcaption>Fig1. - Trulli, Puglia, Italy.</figcaption>

</figure>

[HTML5 New Features??](#)

HTML5, the fifth version of HTML, introduces several new features that significantly enhance the creation of both static and dynamic websites. These features provide greater flexibility, improved performance, and better user experiences.

Audio and Video Support

HTML5 introduces the <audio> and <video> tags, allowing developers to embed audio and video directly into web pages without relying on third-party plugins like Adobe Flash. This makes adding multimedia content faster and more efficient.

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
<title>Example of Video and Audio Tags</title>
```

```
</head>
```

```
<body>
```

```
<h2>Example of video and audio tag</h2>
```

```
<video width="300" height="200" controls autoplay>
```

```
<source src="/html5/foo.ogg" type="video/ogg" />
```

```
<source src="/html5/foo.mp4" type="video/mp4" />
```

Your browser does not support the video element

```
</video>
```

```
<audio controls autoplay>
```

```
<source src="/html5/audio.ogg" type="audio/ogg" />
```

```
<source src="/html5/audio.wav" type="audio/wav" />
```

Your browser does not support the audio element

```
</audio>
```

```
</body>
```

</html>

Vector Graphics

HTML5 supports vector graphics through the <svg> tag, which allows for scalable and interactive graphics. This reduces the need for external plugins and improves performance¹.

```
<svg id="svgelem" height="200" xmlns="http://www.abc.org/2000/svg"
>

<circle id="redcircle" cx="50" cy="50" r="50" fill="red" />

</svg>
```

Semantic Elements

HTML5 introduces new semantic elements like <header>, <footer>, <nav>, <article>, and <section>. These elements provide better structure and meaning to web pages, improving accessibility and SEO².

```
<!DOCTYPE html>

<html>

<head>

<title>HTML Header</title>

</head>

<body>
```

```
<header>
<h1>This is the heading.</h1>
<h4>This is the sub-heading.</h4>
<p>This is the metadata.</p>
</header>
</body>
</html>
```

Form Enhancements

HTML5 enhances forms with new input types (e.g., email, date, time, url, number) and attributes (e.g., placeholder, autofocus, required). These improvements make form handling and user input validation more robust¹.

```
<!DOCTYPE html>
<html>
<body>
<form>
<input type="text" name="fname" placeholder="First name">
<input type="text" name="lname" placeholder="Last name">
<input type="submit" value="Submit">
</form>
</body>
```


</html>

Web Storage

HTML5 introduces localStorage and sessionStorage, providing a way to store data on the client side more securely and efficiently than cookies².

Offline Web Applications

The Application Cache (AppCache) allows web applications to run offline by caching the application files, enhancing performance and availability¹.

WebSockets

WebSockets enable real-time, two-way communication between a client and a server with less overhead than traditional HTTP protocols².

```
var socket = new WebSocket('ws://localhost:8080');

socket.onopen = function() {
  console.log('WebSocket connection opened');
};

socket.onmessage = function(event) {
  console.log('WebSocket message received:', event.data);
};

socket.onclose = function(event) {
  console.log('WebSocket connection closed:', event.code, event.reason);
};
```

```
socket.onerror = function(error) {  
  console.error('WebSocket error:', error);  
};  
socket.send('Hello, server!');
```

[What is Geolocation API? What is drag and drop API? in html ??](#)

Conclusion

HTML5 brings a plethora of new features that enhance web development by providing better multimedia support, improved form handling, and real-time communication capabilities. These features make HTML5 a powerful tool for creating dynamic and interactive website

Geolocation API and **Drag and Drop API** are two powerful features of HTML5 that enhance user interaction and functionality on web pages. Here's a brief explanation of each:

Geolocation API

The **Geolocation API** allows web applications to access the geographic location of a user. This can include latitude, longitude, altitude, and more. It is commonly used in applications like maps, location-based services, and weather apps.

- **Key Features:**

- Retrieves the user's current location.
- Requires user consent for privacy reasons.
- Provides location data in real-time.

- **Example Usage:**

JavaScript

Copy code

```
if (navigator.geolocation) {  
    navigator.geolocation.getCurrentPosition(function(position) {  
        console.log("Latitude: " + position.coords.latitude);  
        console.log("Longitude: " + position.coords.longitude);  
    });  
} else {  
    console.log("Geolocation is not supported by this browser.");  
}
```

Drag and Drop API

The **Drag and Drop API** enables users to drag elements from one location and drop them into another within a web page. It is widely used in file uploads, interactive interfaces, and custom UI designs.

- **Key Features:**

- Allows elements to be draggable.
- Supports events like dragstart, dragover, drop, etc.
- Simplifies creating interactive and dynamic web applications.

- **Example Usage:**

Html

Copy code

```
<div id="dragItem" draggable="true" ondragstart="drag(event)">Drag me!</div>
```

```
<div id="dropZone" ondragover="allowDrop(event)"  
ondrop="drop(event)">Drop here</div>
```

```
<script>
```

```
function allowDrop(event) {  
    event.preventDefault();  
}
```

```
function drag(event) {  
    event.dataTransfer.setData("text", event.target.id);  
}
```

```
function drop(event) {  
    event.preventDefault();  
    const data = event.dataTransfer.getData("text");  
    event.target.appendChild(document.getElementById(data));  
}
```

```
</script>
```

The HTML <tfoot> tag ??

The **HTML <tfoot> tag** is used to give a **footer** group of content in an HTML table. This tag is used in an HTML table with a **header** and **body** which is known as "thead" and "tbody". [<tfoot>](#) tag is child tag of table and parent tag of [<tr>](#) and [<td>](#).

The <tfoot> element must have one or more [<tr>](#) tags inside.

GeeksforGeeks

tfoot Tag

Name	User Id
Ram	@ram_b
Shashank	@shashankla
Rahman	@rahamD
Total user 4	

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<center>
```

```
<h1>GeeksforGeeks</h1>
```

```
<h2>tfoot Tag</h2>
```

```
<table>
```

```
<thead>
```

```
<tr>
```

```
<th>Name</th>
<th>User Id</th>
</tr>
</thead>
<tbody>
<tr>
<td>Ram</td>
<td>@ram_b</td>
</tr>
<tr>
<td>Shashank</td>
<td>@shashankla</td>
</tr>
<tr>
<td>Rahman</td>
<td>@rahamD</td>
</tr>
</tbody>

<!-- tfoot tag starts from here -->
<tfoot>
<tr>
<td>Total user</td>
<td>4</td>
</tr>
</tfoot>
```

```
<!-- tfoot tag ends here -->
```

```
</table>
```

```
</center>
```

Creating Anchor Tag Bookmarks in HTML ???

Anchor tags in HTML (<a>) can be used to create bookmarks, allowing users to navigate directly to specific sections of a webpage. This is particularly useful for long pages where quick navigation is needed.

How to Create a Bookmark

To create a bookmark, you need to define a target section using the id attribute and then link to it using the href attribute of an anchor tag.

Example: Bookmark Within the Same Page

```
<!-- Define the target section with an id -->
```

```
<h2 id="section1">Section 1</h2>
```

```
<!-- Create a link to the target section -->
```

```
<a href="#section1">Go to Section 1</a>
```

When the link is clicked, the page will scroll to the section with the id="section1".

Example: Bookmark on Another Page

<!-- Link to a specific section on another page -->

Go to Section 1 on another page

HTML DOM Events

123

HTML DOM events are actions or occurrences that happen in the browser, which can be detected and handled using JavaScript. These events allow developers to create interactive and dynamic web applications by responding to user interactions or changes in the environment.

Common HTML DOM Events??

Here are some common HTML DOM events and their descriptions:

- **click**: Triggered when an element is clicked.
- **dblclick**: Triggered when an element is double-clicked.

- **mouseover**: Triggered when the mouse pointer is moved onto an element.
- **mouseout**: Triggered when the mouse pointer is moved out of an element.
- **keydown**: Triggered when a key is pressed down.
- **keyup**: Triggered when a key is released.
- **load**: Triggered when an object has loaded.
- **unload**: Triggered when a page has unloaded.
- **focus**: Triggered when an element gains focus.
- **blur**: Triggered when an element loses focus.
- **change**: Triggered when the content of a form element changes.
- **submit**: Triggered when a form is submitted.

inline Events vs JS Event Listeners?

The choice between inline events and JS event listeners in JavaScript depends on the developer's preference and the specific requirements of the application. Inline events are simpler and can be overwritten, while event listeners offer more flexibility and can be attached multiple times for the same event. Modern web development often favors event listeners due to their ability to manage multiple event handlers and maintain cleaner code.

Why CSS is placed in the <head> and JavaScript at the bottom of the <body> in HTML: ??

CSS in the <head>

1. Ensures Proper Styling Before Rendering:

- CSS is critical for styling the page. Placing it in the <head> ensures that the browser loads and applies styles before rendering the content. This prevents a "flash of unstyled content" (FOUC), where users briefly see unstyled HTML.

2. Builds the Render Tree Efficiently:

- The browser builds the **CSSOM (CSS Object Model)** and **DOM (Document Object Model)** simultaneously. Once both are ready, the **Render Tree** is created, and the page is painted. Delaying CSS can block this process, slowing down rendering.

3. Progressive Rendering:

- Placing CSS in the <head> allows browsers to progressively render the page as it loads, improving the user experience by displaying content faster.

JavaScript at the Bottom of the <body>

1. Avoids Blocking HTML Parsing:

- JavaScript execution blocks the browser from parsing HTML. If placed in the <head>, it can delay the loading of the page's content. By placing it at the bottom, the browser can fully load and render the HTML before executing JavaScript.

2. Improves Page Load Speed:

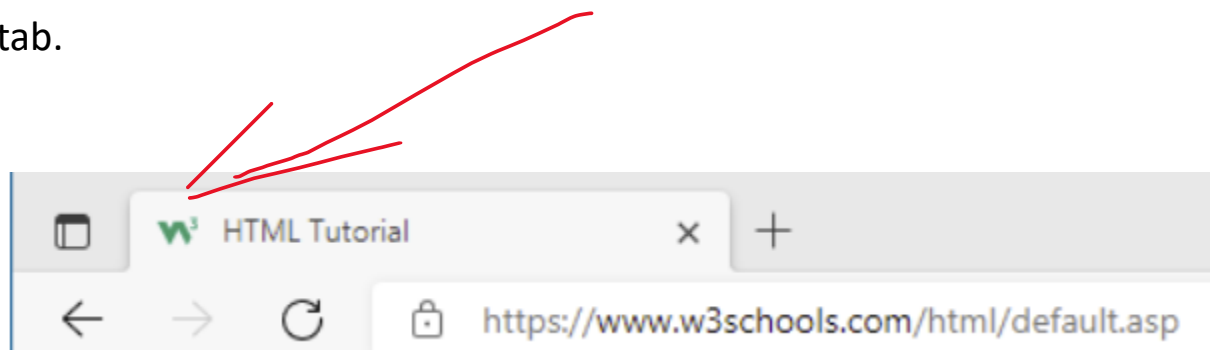
- By deferring JavaScript execution until after the HTML and CSS are loaded, the page appears faster to users, even if the JavaScript takes time to load.

3. Reduces Render Blocking:

- JavaScript can modify the DOM and CSSOM. If loaded too early, it can interrupt the rendering process. Placing it at the bottom ensures the page is already rendered before JavaScript runs.

Favicon ??

A favicon is a small image displayed next to the page title in the browser tab.



```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<title>My Page Title</title>
```

```
<link rel="icon" type="image/x-icon" href="/images/favicon.ico">
```

```
</head>  
<body>
```

HTML Boilerplate ??

HTML Boilerplate refers to a pre-written set of HTML, CSS, and sometimes JavaScript code that serves as a starting template for web development projects. It provides the foundational structure and essential elements needed to kickstart a project, saving time and ensuring consistency.

Key Features of an HTML Boilerplate:

- **Doctype Declaration:** Ensures the document is rendered in standards mode.
- **Meta Tags:** Includes metadata like character encoding, viewport settings for responsiveness, and SEO-related tags.
- **CSS Link:** A placeholder or link to external stylesheets.
- **JavaScript Integration:** Script tags for linking JavaScript files or libraries.
- **Basic Structure:** Includes essential tags like `<html>`, `<head>`, and `<body>`.

What is the difference between `<script>` with `async` and `defer` in html??

The Difference Between `<script>` with `async` and `defer` in HTML

When including JavaScript in an HTML document, the `async` and `defer` attributes control how and when the script is loaded and executed. Here's a concise breakdown:

1. `async` Attribute

- **Loading Behavior:** The script is downloaded in parallel with the HTML parsing.
- **Execution Timing:** The script is executed **as soon as it is downloaded**, potentially interrupting HTML parsing.
- **Use Case:** Best for scripts that are independent of other scripts or the DOM (e.g., analytics or tracking scripts).

2. `defer` Attribute

- **Loading Behavior:** The script is also downloaded in parallel with the HTML parsing.
- **Execution Timing:** The script is executed **after the HTML parsing is complete**, just before the `DOMContentLoaded` event.
- **Use Case:** Ideal for scripts that depend on the fully parsed DOM or need to maintain execution order with other `defer` scripts.

[HTML Custom Data Attributes](#) ???

HTML **custom data attributes** (data-*) allow you to store extra information directly on HTML elements without affecting their rendering or requiring non-standard attributes. They are part of the HTML5 specification and are valid on **any HTML element**.

A data attribute name must start with data- followed by a lowercase, hyphen-separated name, and its value can be any string.

Example in HTML:

```
<ul>
<li data-animal-type="bird">Owl</li>
<li data-animal-type="fish">Salmon</li>
<li data-animal-type="spider">Tarantula</li>
</ul>
```

Accessing in JavaScript: You can retrieve and modify data attributes using the **dataset** property, which automatically converts hyphenated names to camelCase.

```
const item = document.querySelector('li[data-animal-type="bird"]');
```

```
// Read value
```

```
console.log(item.dataset.animalType); // "bird"
```

```
// Update value  
item.dataset.animalType = "eagle";
```

You can also use **getAttribute()** and **setAttribute()**:

```
console.log(item.getAttribute('data-animal-type')); // "eagle"  
item.setAttribute('data-animal-type', 'hawk');
```

Using in CSS: Data attributes can be targeted in CSS selectors or used with the attr() function for generated content.

```
/* Style based on attribute value */  
li[data-animal-type="fish"] {  
  color: blue;  
}
```

```
/* Display attribute value before element */  
li::before {  
  content: attr(data-animal-type) " - ";  
}
```

All
Codeing
Done
AT
DUCAT ..


```
<!doctype html>
```

```
<html>
```

```
<head><title></title>
```

```
<style>/* COMMENT TAG FOR CSS */ </style>
```

```
<!-- <script>COMMENT TAG FOR js</script> -->
```

```
</head>
```

```
<body>
```

```
<!--commnet tag for html-->
```

```
<!--#####HEADEING TAGS :STARTS FROM H1 TO H6:-
```

```
EX:<H1>EXAMPLE FOR HEDEING</H1>-->
```

```
<!-- ##### IMAGE TAGS:- -->
```

```
<!-- 1.online image: -->
```

```
<!-- syntax: <img src="" title="" alt="" height="200"weidth=""/>
```

```
 -->
```

```
<!-- 2. offline image:
```

```
 -->
```

<!--####LISTS:-

1.ORDER LIST-

SYNTAX:

 lists item

 lists item

2.<UNORDER LIST:

SYNTAX:

lists item

lists item

3.DISCRPTION LISTS:

<dl>

<dt>dt-define the term.</dt>

<dd>dd-define each terms.</dd>

</dl> -->

<!-- #####FOEMATEING HTML TAGS:

1.used for bold the text

2.used for bold the text

3.<i>text to italic.</i>

4.text cahnges

5.<mark>heighlight the text .</mark>

6.<ins>underline the content . </ins>

7.cross the text .

8.<small>make the text small .</small>

9.<big>make the text big .</big>

10.<p> subTAG ex: H₂O <p>

11.<p>2³=8</p>-->

<!-- # #####BLOCK LEVEL ELEMENTS && INLINE LEVEL ELEMENTS:

1.BLOCK LEVEL ELEMENTS:tags which always starts from the new line and from browser side

add space between top and bottom. ex:

<h1>headeing</h1>

<p>paragraph</p>

<div>DIV TAG:</div>

<hr>headeing row TAG</hr>

2.INLINE LEVEL ELEMENTS: TAGS THAT DOES NOT START FROM THE NEW .EX:

SPAN TAG

anchor tag: redirect to another page. ex: a href TAG.

```
<a href="http://www.amazon.in">amazon</a>
```

```
<a href="http://www.amazon.in" target="_blank">amazon</a>
```

```
<a href="http://www.amazon.in" target="_self">amazon</a>
```

```
-->
```

```
<!-- #####HORIZONTAL TABLE:with colspan-->
```

```
<!-- <table border="2">
```

```
    <tr>
```

```
        <th>name</th>
```

```
        <th colspan="2">class</th>
```

```
        <th>roll.no</th>
```

```
    </tr>
```

```
    <tr>
```

```
        <td>dhruv</td>
```

```
        <td>HTML</td>
```

```
        <td>CSS</td>
```

```
        <td>123</td>
```

```
    </tr>
```

```
</table> -->
```

```
<!-- vartical table with rowspan-->
```

```
<!-- <table border="2">
```

```

<tr>
  <th>name</th>
  <td>dhruv</td>
</tr>
<tr>
  <th rowspan="2">roll no:</th>
  <td>123xx</td>
</tr>
<tr>
  <td>123xx</td>
</tr>
<tr>
  <th>class</th>
  <td> 12<td>
  </tr>

```

</table> -->

<!-- ##### html quotations:

1.blockquote: //for programer refrence not for the user refrence.

```

<blockquote
cite="https://www.w3schools.com/html/html_styles.asp">xyz
paragraph here</blockquote>

```

2. q tag:

<p>hello student iam your teacher</p>

<p><q>hello students</q>iam your teacher</p>

3.abbr:

hello student iam your **teacher**

4.address:

<p>Address:soth.ex new delhi</p>

```
<address> soth.ex new delhi </address>
```

5. cite tag:

<p><cite>this is cite tag </cite></p>

6.bdo tag: //used to reverse-->

```
<!-- <p><bdo dir="rtl">hello students</bdo></p> -->
```

```
<!-- 7. # iframe tag:
```

this tag is used to open a web page within a web page.

```
<iframe src="https://www.flipkart.com/"title="amazon" height="700"
weidth="700"
```

```
style="border:5px;"></iframe>-->
```

```
<!--#####html entites
```

ex:

<p>hello world</p>

<p>hello world</p>

<p>hello &nb sp; world</p>

<p>imogies ex:🗼</p>-->

```
<!--#####FORM
```

PLACE HOLDER: DEFINE THE VALUE YOU SHOULD PUT IN THE BOX.

NAME = JUST LIKE THE NAME GIVEN TO PERSON FOR IDENTIFICATION.

VALUE= DEFAULT VALUE PROVIDE BY PROGRAMER.

disable =is used to disable user input..

action= it will redirect you to another html document by clickeing on submit button.

enctype= "multipart/form-data" :- it is a kind type of data used to send form to server.

„used with `<input type = file />..`

method =get :- contain data for public view in the form of url.

method = post:- contain private type of data.-->

```
<!-- <form enctype="multipart/form-data" method="post"> -->
```

```
<!-- <form action="/a.html" enctype="multipart/form-data"
method="get">
```

```
<input type = "text" placeholder = "enter name." name = "sname" value = "dhruv" disabled /><br>
```

```
<input type = "email" placeholder = "enter email"
name = "email" value = "ronacksingh@gmail.com" /><br>
```

<input type = "submit" value="login"/>

<input type="date">

<input type="datetime">

<input type="file">

HIDDEN :<input type="hidden">

<input type="time">

<input type="range">

<input type="text">

<input type = "radio" id="html" name="fav language" value="HTML">

<label for="html">HTML</label>

<input type = "radio" id="css" name="fav language" value="CSS">

<label for="css">CSS</label>

<input type = "checkbox" id="yes" name="fav language" value="YES">

<label for="yes">YES</label>

<input type = "checkbox" id="no" name="fav language" value="NO">

<label for="no">NO</label>

<input type="reset">

</form> -->

<!--#####video & audio:

<video controls autoplay muted height="200"weidth="200">


```
<source src="./fast.mp4"type="video/mp4"/>
</video>
<audio controls autoplay muted height="200"weidth="200">
  <source src="./fast.mp4"type="audio/mp4"/>
</audio>-->
<script>
//  //local stogarge:-
//  localStorage.clear();
//  localStorage.setItem("student1","dhruv");
//  //localStorage.setItem("student2","xyz");
//  var getdata= localStorage.getItem("student1");
//  console.log(getdata);
//  localStorage.removeItem("student1");
//session stogarge:-
//  sessionStorage.clear();
//  sessionStorage.setItem("student1","dhruv");
//  sessionStorage.setItem("student2","xyz");
//  var getdata= sessionStorage.getItem("student1");
//  console.log(getdata);
//  sessionStorage.removeItem("student2");
```

</script>

</body>

</html>